

Nash DRL

Document Version	12
Last Modified	2026/08/22
Change Log	New: • Section 4: Network Design Edited: • Section 3.2.1: Input State Model (Data Structure) Removed: • Nothing

1 Introduction

1.1 Our Research Focus

The main problem we focus on is providing a solution for reducing or eliminating hard constraint violations in the autonomous multi-agent electrical vehicle path planning problem. Hard constraint violations occur in such systems because the algorithms that control the autonomous agents' behavior—in our example, the path planning algorithms of EAVs—focus solely on maximizing the total system payoff (total reward).

By concentrating on game-theoretic approaches, our goal is to show that we can design these algorithms in such a way that hard constraint violations do not occur, while the total system reward may or may not decrease.

1.2 Problem Definition

In an environment composed of heterogeneous Electric Autonomous Vehicles (EAVs) with different driving behaviors, some vehicles exhibit normal driving behavior while others exhibit abnormal driving behavior that causes traffic congestion and consequently increases the total travel time of all vehicles.

At the same time, the charging price at each charging station is determined by demand. In this scenario, if vehicles form coalitions so that they share more common paths during their trips, the demand for charging on those roads rises, which in turn drives the price down — a beneficial outcome for all vehicles. We define a coalition as a set of players (vehicles) that share a resource (roads) to achieve a better result than when they do not share resources. All vehicles face two types of constraints. The first is travel time, which we treat as a soft constraint (violating it is tolerable). The second is a budget constraint, which is a hard constraint and cannot be violated.

Assume an algorithm handles path planning for the EAVs, and that vehicles with normal driving behavior outnumber those with abnormal behavior. In that case, the path-planning algorithm converges to a situation where vehicles with abnormal behavior are sacrificed for the greater good. More specifically, the algorithm decides to separate the abnormal-behavior vehicles onto different routes, imposing higher charging costs on them (because of low charging demand on those routes), which in turn causes some of them to violate their hard budget constraints. Meanwhile, the vehicles with normal driving behavior enjoy a substantial improvement in travel time and only a slight increase in charging cost. Thus, by sacrificing a subset of vehicles, the overall system payoff (reward) becomes greater than in the situation where the algorithm chooses not to sacrifice those abnormal-behavior vehicles.

1.3 Illustrative (Motivational) Examples

Symbol	Definition
e	An individual edge in the road network.
l_e	Length of edge e [km].
f_e	Total vehicular flow on edge e
$p_e(f_e)$	Unit energy price at the charging station on edge e , as a function of flow
v_i	Free-flow speed of vehicle i [km/h].
C_{v_i}	Total fuel cost for vehicle i
T_{v_i}	Total travel time for vehicle i
R_{v_i}	Total reward for vehicle i
TR	System total reward
η	Vehicle energy consumption rate [kWh/km].
α, β	Congestion function parameters
P	Threshold penalty parameter in the reward function
w_T, w_C	Weight coefficient for travel time and fuel cost in the reward function

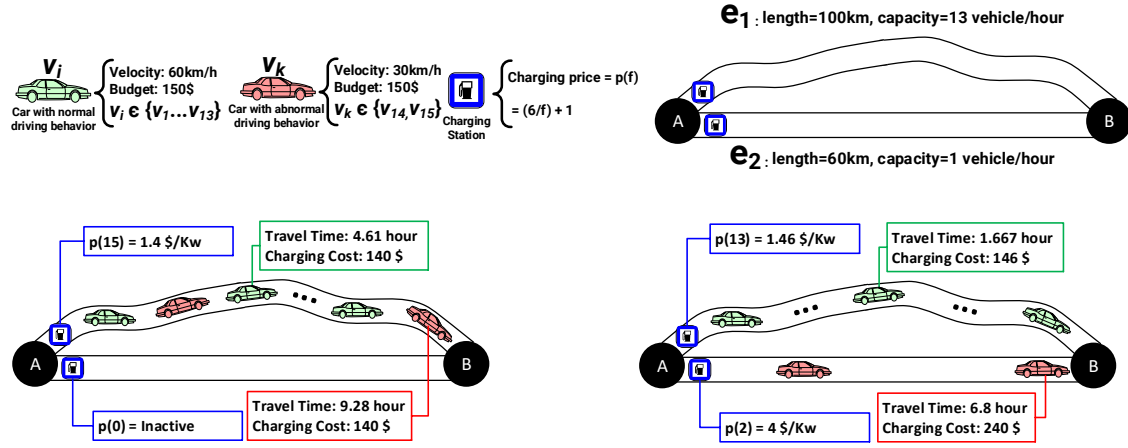


Figure 1: An Illustrative Example. (Up) Road topology, vehicles and charging station configuration. (Left) Scenario 1, Coalition formed, caused no hard constraint violation. (Right) Scenario 2, Without coalition caused hard constraint violation for red vehicles in terms of their budgets.

Every edge hosts a charging station with identical parameters: overhead $O = 6$, fixed unit cost $C=1$, floor price $p_e^{min} = 0$, giving the flow-dependent price (based on the formula 3):

$$p_e(f_e) = \max_e \left(0, \frac{6}{f_e} + 1 \right)$$

Vehicles consume energy at $\eta = 1 \text{ kWh/Km}$, congestion follows (formula 5) with $\alpha = 0.15$, $\beta = 4$, and rewards are computed by formulas (7)-(8) with $w_T = 1$, $w_C = 1$ and a hard-budget penalty $P = 100$ (i.e., a vehicle that exceeds its budget receives -100 regardless of time and cost).

Comparison

The split solution, which sacrifices the abnormal-behaved vehicles, achieves a higher total system reward (−2122.83 vs. −2178.43), but at the cost of forcing those vehicles to violate their budgets. The grand-coalition solution eliminates every hard-constraint violation, yet the normal vehicles suffer a travel-time penalty that drives the system reward lower.

Scenario 1 Calculations – Grand Coalition (no sacrifice).

To keep the abnormal vehicles within their tight budget, all 15 vehicles must concentrate on the edge e_1 , where the high flow drives down the unit price:

$$p_{e_1}(f_{e_1}) = \max_{e_1} \left(0, \frac{6}{f_{e_1}} + 1 \right) = \left(\frac{6}{15} + 1 \right) = 1.4 \text{ \$/Kw} \rightarrow C_{v_i} = 100 \times 1.4 = 140 \text{ \$}$$

The total traveling time for normal vehicles would be (based on formula (5)):

$$T_{v_i} = \frac{l_e}{v_i} \left(1 + \alpha \left(\frac{f_e}{cap_e} \right)^\beta \right) = \frac{100}{60} \left(1 + 0.15 \left(\frac{15}{13} \right)^4 \right) = 4.61 \text{ Hour}$$

And for abnormal vehicles would be:

$$T_{v_k} = \frac{100}{30} \left(1 + 0.15 \left(\frac{15}{13} \right)^4 \right) = 9.25 \text{ Hour}$$

Consequently, the total reward of system is (formulas (7) and (8)):

$$R_{v_i} = -w_T T_{v_i} - w_C C_{v_i} = -4.61 - 140 = -144.61$$

$$R_{v_k} = -9.25 - 140 = -149.25$$

and the total system reward is:

$$TR = 13 \times (-144.61) + 2 \times (-149.25) = -2178.43$$

with **zero hard-constraint violations**.

1.3.1 Scenario 2 Calculations – Split (sacrifice)

The 13 normal vehicles avoid the direct road and take the highway (edges e_1):

$$p_{e_1}(13) = \left(\frac{6}{13} + 1 \right) = 1.46 \text{ \$/Kw} \rightarrow C_{v_i} = 100 \times 1.46 = 146 \text{ \$}$$

$$p_{e_2}(2) = \left(\frac{6}{2} + 1 \right) = 4 \text{ \$/Kw} \rightarrow C_{v_k} = 60 \times 4 = 240 \text{ \$}$$

Total traveling time for vehicles would be:

$$T_{v_i} = \frac{100}{60} \left(1 + 0.15 \left(\frac{13}{13} \right)^4 \right) = 1.91 \text{ Hour}$$

And for abnormal vehicles would be:

$$T_{v_k} = \frac{60}{30} \left(1 + 0.15 \left(\frac{2}{1} \right)^4 \right) = 6.8 \text{ Hour}$$

Consequently, the total reward of each vehicle is (formula (7)):

$$R_{v_i} = -1.91 - 146 = -147.91$$

$$R_{v_k} = -100$$

and the system reward is

$$TR = 13 \times (-147.91) + 2 \times (-100) = -2122.83$$

with **two hard-constraint violations**.

2 Problem Formulation

2.1 Road Network Model

A directed graph $G = (V, E)$. Each edge $e \in E$ has

- length l_e [km],
- capacity c_e [vehicles/h].

2.2 Vehicles Model

A set $I = \{1, 2, \dots, N\}$ composed of heterogenous vehicles, with different driving behavior. Each vehicle i has:

- origin O_i and destination D_i ,
- free-flow speed v_i [km/h],
- budget B_i [monetary units]

For each vehicle i , a path π_i (a sequence of edges from O_i to D_i) encoded by binary variables:

$x_{i,e} = \begin{cases} 1, & \text{if edge } e \in \pi_i \\ 0, & \text{otherwise} \end{cases}$	(1)
---	-----

The flow on edge e is:

$f_e = \sum_{i \in I} x_{i,e}$	(2)
--------------------------------	-----

2.3 Charging Stations and Fuel Cost Model

On each edge e there is a charging station. The station provides energy at a unit price that decreases with the number of vehicles using that edge (the demand):

$$p_e(f_e) = \max_e \left(p_e^{\min}, \frac{O}{f_e} + C \right) \quad (3)$$

Where:

- $O > 0$ station overhead cost,
- $C \geq 0$ is a fixed unit cost,
- $p_e^{\min} > 0$ is a floor price of station e . (If $f_e = 0$ the station is inactive; the price for an unused edge is irrelevant.)

Vehicles consume energy at a constant rate η [kWh/km]. Vehicle i must purchase the energy required for its entire path from the stations on the edges it traverses (pay-as-you-go). Total fuel cost for vehicle i :

$$C_i = \eta \sum_{e \in E} x_{i,e} l_e p_e(f_e) \quad (4)$$

2.4 Travel Time and Congestion Model

Travel time of vehicle i on edge e depends on its own free-flow speed and on the total flow f_e via a standard congestion function:

$$\tau_{i,e}(f_e) = \frac{l_e}{v_i} \left(1 + \alpha \left(\frac{f_e}{c_e} \right)^\beta \right), \alpha \geq 0, \beta \geq 1. \quad (5)$$

Total travel time for vehicle i :

$$T_i = \sum_{e \in E} x_{i,e} \tau_{i,e}(f_e) \quad (6)$$

2.5 Reward Model

Vehicle total reward:

$$R_i = \begin{cases} -w_T T_i - w_C C_i, & C_i \leq B_i \\ -P, & C_i > B_i \end{cases} \quad (7)$$

System total reward:

$$TR = \sum_{i \in I} R_i \quad (8)$$

3 State and Action Model (Data Structure)

3.1 Map (road network)

We use a **Compressed Sparse Row (CSR)**¹ format combined with parallel Edge Property Arrays. CSR is the industry standard for graph neural networks and large-scale routing because it stores adjacency in three compact, cache-friendly arrays without any Python list overhead.

3.1.1 Map Data Structure

The map is a directed graph $G = (V, E)$. Each edge has a length l_e (float) and a capacity c_e (float). As states before, we store the graph in **CSR** format.

- **row_ptr**: An integer array of length $V + 1$. **row_ptr**[u] is the start index in **col_idx** and **edge_ids** for the outgoing edges of node u . **row_ptr**[$u+1$] is the exclusive end index.
- **col_idx**: An integer array of length E . Stores the destination node IDs for each outgoing edge, sorted sequentially by source node.
- **edge_ids**: An integer array of length E . Stores the unique Edge ID corresponding to each entry in **col_idx**.
- **edge_len**: A float array of length E storing the length l_e [km] (indexed by Edge ID).
- **edge_cap**: A float/int array of length E storing the capacity c_e [vehicles/h] (indexed by Edge ID).
- **edge_flow**: A float array of length E storing the current total flow f_e (updated dynamically in the environment step, indexed by Edge ID).
- **edge_from**: A float/int array of length E to quickly get the source node of an edge without reverse-lookup.

3.1.2 Numerical Example

Assume a simple directed graph with 4 nodes (0, 1, 2, 3) and 5 directed edges.

<i>Edge ID</i>	<i>From (u)</i>	<i>To (v)</i>	<i>Length (km)</i>	<i>Capacity (veh/h)</i>
0	0	1	10.0	100
1	1	3	20.0	80
2	0	2	15.0	120
3	2	3	25.0	60
4	1	2	5.0	50

CSR Construction Logic:

To build the CSR, we group outgoing edges by their source node, sorting them by destination ID for deterministic ordering.

¹ [https://en.wikipedia.org/wiki/Sparse_matrix#Compressed_sparse_column_\(CSC_or_CCS\)](https://en.wikipedia.org/wiki/Sparse_matrix#Compressed_sparse_column_(CSC_or_CCS))

- **Node 0:** Outgoing to 1 (Edge 0), 2 (Edge 2) $\rightarrow [1, 2]$
- **Node 1:** Outgoing to 3 (Edge 1), 2 (Edge 4) $\rightarrow$ Sorted by dest: $[2, 3]$
- **Node 2:** Outgoing to 3 (Edge 3) $\rightarrow [3]$
- **Node 3:** No outgoing edges $\rightarrow []$

Data storage (CSR Arrays):

- `row_ptr` = $[0, 2, 4, 5, 5]$
(Explanation: Node 0 starts at 0, Node 1 starts at 2, Node 2 starts at 4, Node 3 starts at 5, end is 5)
- `col_idx` = $[1, 2, 2, 3, 3]$
(Explanation: Node 0 $\rightarrow [1,2]$; Node 1 $\rightarrow [2,3]$; Node 2 $\rightarrow [3]$)
- `edge_ids` = $[0, 2, 4, 1, 3]$
(Explanation: Maps `col_idx` to edge IDs. Edge 0 goes to 1, Edge 2 goes to 2, Edge 4 goes to 2, Edge 1 goes to 3, Edge 3 goes to 3).

Edge Properties (Indexed by Edge ID):

- `edge_len` = $[10.0, 20.0, 15.0, 25.0, 5.0]$
- `edge_cap` = $[100, 80, 120, 60, 50]$
- `edge_flow` = $[0.0, 0.0, 0.0, 0.0, 0.0]$ (initialized to 0, updated during simulation).

3.2 The Input State (Global Observation for the Neural Network)

Since the DRL algorithm uses a global state to output global actions, we flatten all vehicle features into a single tensor. To maintain Permutation Invariance (as per the original Nash-DQN), we separate features into "Global" (non-invariant) and "Per-Agent" (which will be aggregated via Deep Sets).

Each vehicle has predefined set of set of trips (each trip defined with one origin and destination), such that destination of current trip considered as origin of next trip and so on. The algorithm should determine the path of each trip in each step of algorithm until the whole trips (episode) done. Vehicles may have trip sets with different sizes.

3.2.1 Input State Data Structure

- **agent_features:** A 2D matrix of size $[N \times F]$, where N is the number of vehicles and F is the number of features per vehicle.
- **edge_flow_features:** A 1D vector $[E \times 1]$ containing the current flow on every edge (needed to compute congestion costs in the reward, though the environment calculates this, the network might need it for the value estimate. We can just include it in `global_features` or feed it separately).

For a single vehicle i at time step t , the features F must include:

- Current Stop ID (one-hot encoded or normalized integer).
- Next Destination Node ID (destination of the current leg).
- Final Destination ID (destination of final trip)
- Total number of remaining destinations for vehicle v_i
- Remaining Budget B_i^{rem} .

- Free-flow Speed v_i .

3.3 The Output Action (Routes / Trip-Sets for all vehicles)

Instead of outputting a discrete path index, the neural network outputs raw weights for every edge in the graph. Subsequently, a deterministic `action_to_path_mapper` algorithm constructs valid routes (respecting origins, intermediate stops, and destinations) from these weights.

3.3.1 Data Structure

The action output is a matrix of size $[N \times E]$.

- **`action[i][e]`**: Represents the weight assigned to edge with ID **`e`** for vehicle **`i`**.
- These weights are real-valued (can be positive or negative, passed through an activation function depending on the mapper, or raw logits).
- During training, exploration noise can be added directly to these weights (e.g., Gaussian or Dirichlet noise per row).
- A deterministic algorithm **`action_to_path_mapper(weights, current_node, destination)`** takes these weights and the specific vehicle constraints to output the exact path (sequence of edge IDs) for each leg.

Note on the Mapper: The mapper is deterministic and non-trainable. For example, it could interpret the weights as edge costs and run a shortest-path algorithm (e.g., Dijkstra where cost = $1/(\text{weight} + \epsilon)$), or it could greedily select the highest-weighted outgoing edge at each step that makes progress toward the destination. Because the mapper is deterministic, gradients from the reward flow back directly to the $[N \times E]$ weights.

3.3.2 Numerical Example

Assume $N=2$ vehicles and $E=5$ edges.

Network Output Action Matrix (weights):

<i>Vehicle \ Edge</i>	<i>Edge 0</i>	<i>Edge 1</i>	<i>Edge 2</i>	<i>Edge 3</i>	<i>Edge 4</i>
Vehicle 0 (<i>At 0 $\rightarrow$ Dest 3</i>)	1.5	0.2	3.0	0.1	-0.5
Vehicle 1 (<i>At 1 $\rightarrow$ Dest 3</i>)	0.4	2.0	-0.2	1.0	2.5

Processing by `action_to_path_mapper`:

Suppose the mapper uses a greedy approach: "At each step, move along the outgoing edge with the highest weight that gets closer to the destination, avoiding cycles."

- **Vehicle 0** (At Node 0):
 - Outgoing edges: Edge 0 (to 1, weight 1.5) and Edge 2 (to 2, weight 3.0).
 - Highest weight is Edge 2 $\rightarrow$ moves to Node 2.
 - From Node 2, outgoing is Edge 3 (to 3, weight 0.1).
 - Path selected: [Edge 2, Edge 3].
- **Vehicle 1** (At Node 1):
 - Outgoing edges: Edge 1 (to 3, weight 2.0) and Edge 4 (to 2, weight 2.5).

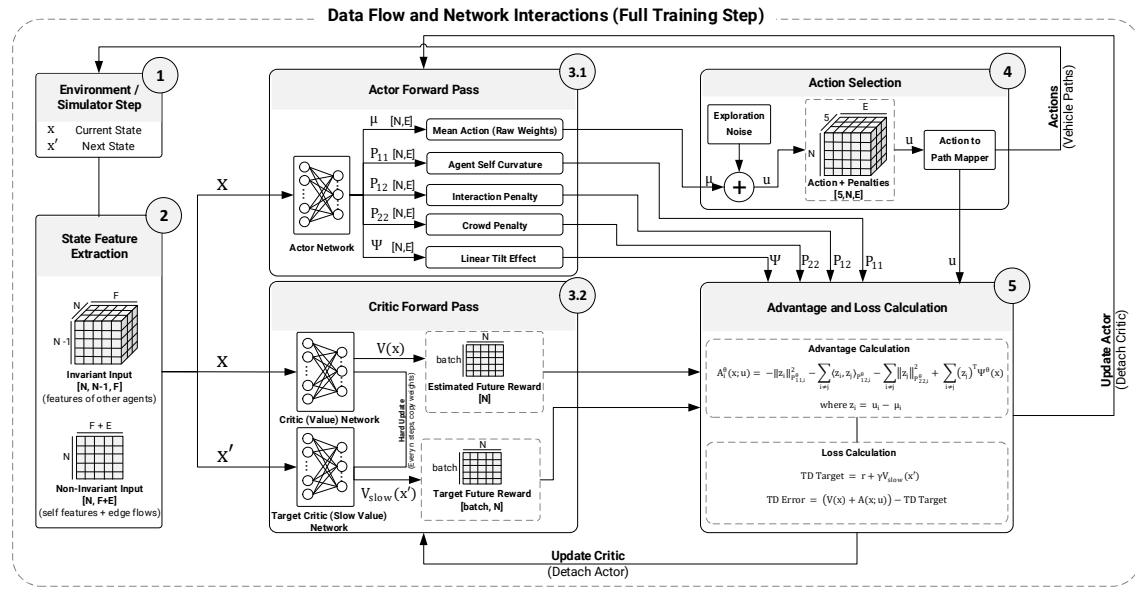
- Since destination is Node 3, Edge 1 reaches it directly. Highest weight is 2.5 for Edge 4, but if it goes there it has to come back. A more complex Dijkstra mapper would likely prefer Edge 1.
- Path selected (assuming Dijkstra): [Edge 1].

The environment receives these edge lists, updates `edge_flow` by adding vehicles to their selected paths, and computes the reward.

4 Network Design

The core of our proposed DRL framework consists of three distinct neural modules: an Actor Network that parameterizes the Locally Linear-Quadratic (LQ) Advantage function, a Critic (Value) Network that estimates the baseline state-value, and a Target Critic Network that ensures training stability. All three networks are built upon fully connected architectures, leveraging the permutation-invariant properties required for multi-agent systems. Specifically, to handle the variable crowd of surrounding vehicles (label invariance), the networks incorporate a Deep Sets aggregation layer, which processes the features of all other agents through a shared embedding function and sums them to produce a consolidated "crowd" representation.

4.1 Data Flow and Network Interactions (Full Training Step)

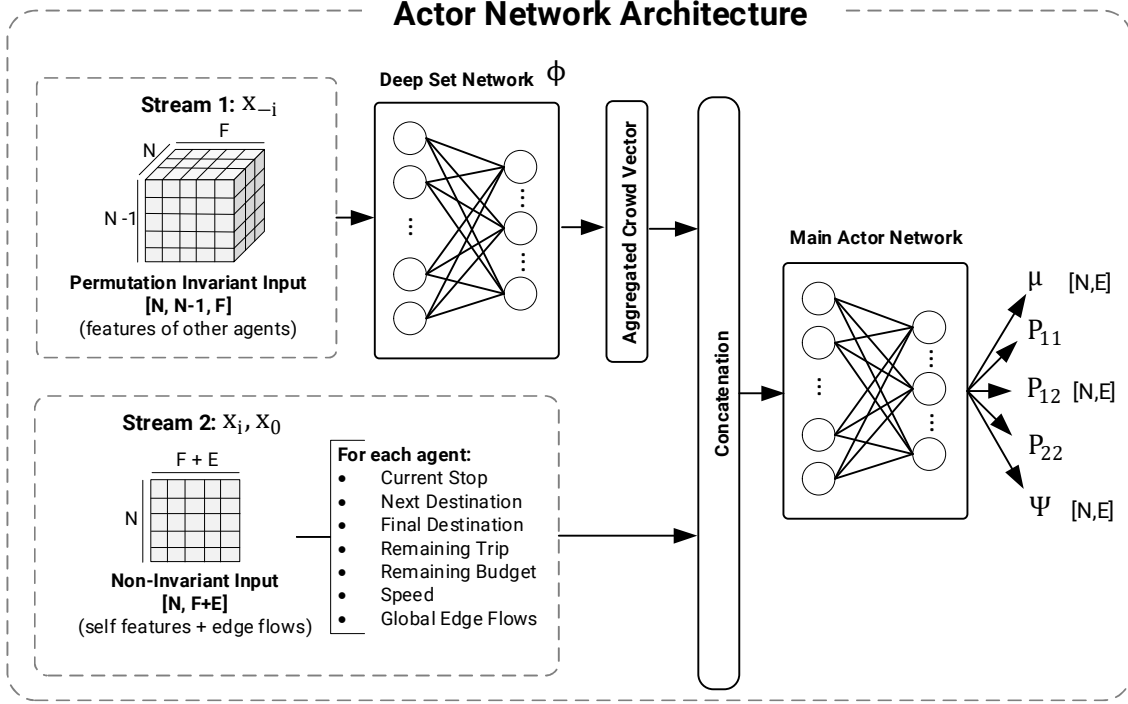


As depicted in the figure, each training iteration begins with the environment simulator advancing from the current state X to the next state X' based on the selected actions. Subsequently, the State Feature Extraction module processes this raw state into two distinct tensor streams to serve the individual network architectures. The first stream constructs the Non-Invariant Input of shape $[N, F + E]$, which stacks the local features of each vehicle (current stop, next destination, remaining budget, etc.) along with the global edge flow features (E). The second stream builds the Invariant Input of shape $[N, N - 1, F]$, which, for each focal agent i , contains the feature vectors of all other agents $j \neq i$. This structure allows the networks to naturally enforce label invariance, as the relative ordering of rivals has no impact on the aggregated features.

Once the features are extracted, the Actor Network processes these tensors to output the LQ parameters (P_{11} , P_{12} , P_{22} , Ψ). The mean action μ is then combined with exploration noise, passed through a deterministic Action-to-Path Mapper, and executed in the environment to yield the reward r and the next state X' . Simultaneously, the Critic Network computes the baseline value $V(x)$ from the current state, while the frozen Target Critic Network computes $V_{slow}(x')$ from the next state. The core of the learning signal is derived from the Advantage Calculation, which uses the Actor's parameters to compute the Advantage $A_i^\theta(x; u)$ using the

LQ formula (where $z_i = u_i - \mu_i$). The Loss Calculation module then computes the Temporal Difference (TD) error by comparing the predicted Q-value ($V(x) + A(x; u)$) against the TD target ($r + \gamma V_{slow}(x')$). This error drives the alternating optimization steps, where the Critic and Actor are updated sequentially while their counterpart outputs are detached to prevent destructive interference.

4.2 Actor Network Architecture

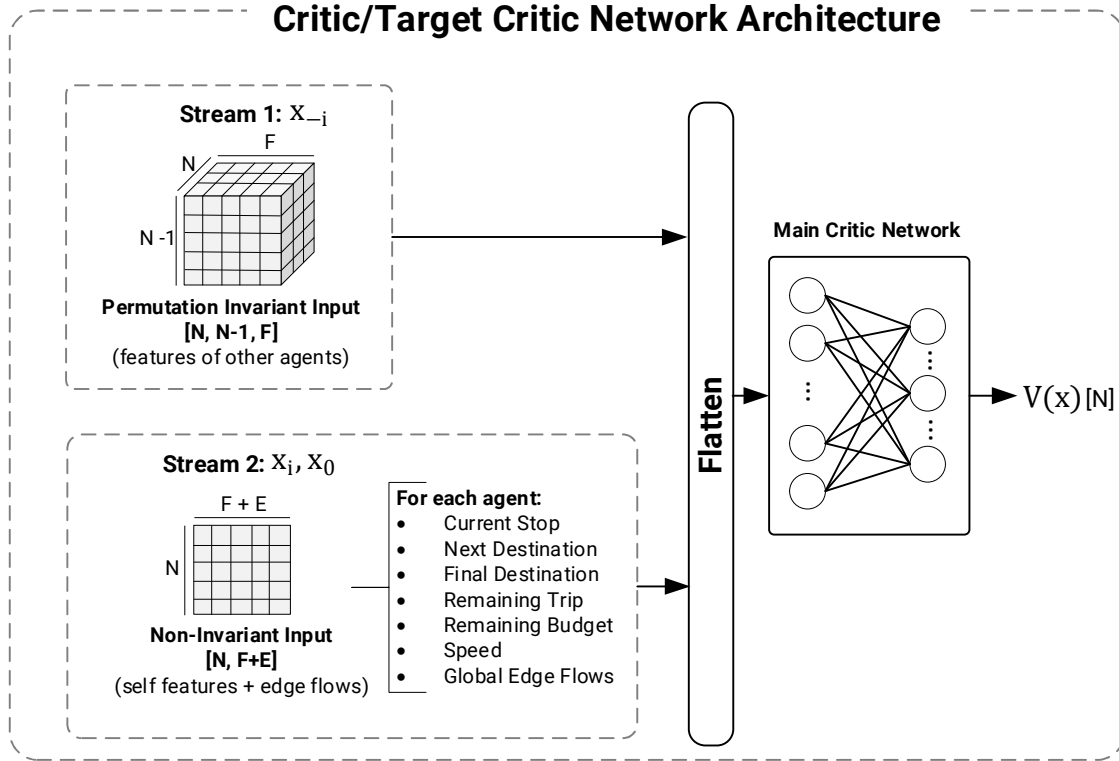


The Actor Network is engineered to directly model the LQ-Advantage function and provide the optimal mean policy. It consists of a Deep-Set Network combined with a main processing trunk to preserve permutation invariance while capturing agent-specific nuances. The architecture is fundamentally divided into two streams: Stream 1 processes the features of other agents X_{-i} (shape $[N, N-1, F]$), while Stream 2 handles the non-invariant features X_i, X_0 (shape $[N, F+E]$), consisting of the ego vehicle's current stop, next stop, final destination, remaining trip count, remaining budget, speed, and the global edge flows.

Within Stream 1 (Deep Sets), each of the $N-1$ rival feature vectors are independently passed through a shared embedding network ϕ . Subsequently, an element-wise summation aggregates these embedded vectors into a single, fixed-size "crowd vector" that is entirely independent of the order of the rivals. This aggregated representation is then concatenated with the output of Stream 2 (the ego and global features) to form a comprehensive latent input. This concatenated vector is fed into the main actor trunk, which comprises four fully connected hidden layers, each containing 32 nodes and activated by the SiLU function. The final output layer of the Actor Network produces a tensor of shape $[5, N, E]$, where the five channels directly correspond to the set of LQ parameters: the mean action μ , the ego penalty P_{11} , the interaction penalty P_{12} , the rival penalty P_{22} , and the linear tilt Ψ . Importantly, the outputs for P_{11} and P_{22} are passed through an element-wise operation to ensure strict positive-definiteness, preserving the concavity requirements of the Nash equilibrium. By directly outputting these parameters,

the network enables the analytical computation of the local Nash equilibrium without requiring iterative game-theoretic solvers.

4.3 Critic and Target Critic Network Architecture



The Critic Network (Value Network) is designed to approximate the scalar baseline value $V(x)$, representing the total expected future reward for each agent under the current joint policy. Mirroring the Actor's structural philosophy, the Critic incorporates a Permutation Invariant Input layer of shape $[N, N - 1, F]$ to aggregate rival information, ensuring that the value estimate is robust to the reordering of heterogeneous vehicles. However, unlike the Actor, the Critic's main processing path extensively utilizes a flattening operation. It transforms the ego vehicle's features—comprising the current stop, next stop, final destination, remaining trips, remaining budget, and speed—alongside the global edge flow vector, into a concise flat representation. These flattened features are then concatenated with the aggregated crowd representation obtained from the permutation invariant layer. This unified vector is subsequently fed into the main value trunk, which consists of four fully connected hidden layers with 32 nodes each and SiLU activations. The output layer generates a single scalar value $V_i(x)$ for each agent i , predicting the baseline future return from the current global state.

The Target Critic Network (Slow Value Network) is an exact architectural replica of the Main Critic Network, designed to provide a stable, slowly evolving target for the TD updates. Its input structure is identical, accepting the next state X' and processing it through the same permutation invariant layer and flattening mechanisms to produce $V_{slow}(x')$. This network's parameters are not updated via backpropagation; instead, they are periodically synchronized with the Main Critic Network through a hard copy. By freezing the target estimates for a fixed number of steps, the Target Critic effectively anchors the TD error calculation, mitigating the "chasing a moving target" instability commonly observed in deep Q-learning. The

asynchronous update schedule of this network is crucial for decoupling the immediate policy improvements (driven by the Actor) from the long-term value estimations (driven by the Critic), ultimately fostering a smoother and more reliable convergence toward the multi-agent Nash equilibrium.